Entrega 3 — Guía de redacción

Capítulos 12, 13 y 14 · Ticket SCRUM-41 · Fecha límite: lunes 19 de octubre de 2026
Plataforma de Gestión Académica y Analítica de Rendimiento Estudiantil · Grupo Adilman · UES-FMO

Por qué esta entrega es la más fácil de las cuatro

Los tres capítulos describen cosas que ya están construidas, verificadas y documentadas en el repositorio. No hay que diseñar nada: hay que explicar lo que existe y justificar por qué se hizo así. Además, el **10 de octubre se volvió a correr la validación completa del data warehouse sobre una máquina distinta** y dio exactamente los mismos 66 resultados que el 5 de octubre, así que todas las cifras de esta guía son reproducibles y están respaldadas por un registro versionado en git.

Como en la guía anterior: aquí tienes la estructura, los datos verificados y párrafos de muestra que enseñan el registro. El texto final lo escribes tú, porque el docente prohíbe generar el documento con IA y porque en la defensa te preguntan por cualquier párrafo.

Material que ya existe y hay que citar

Fuente	Para qué capítulo
`database/oltp/01` a `09`	12 — esquema operacional, reglas, RBAC, costos
`etl/` (10 módulos) y sus docstrings	13 — el pipeline, muy bien documentado en el código
`database/oltp/10`, `12`, `13`, `14`	14 — modelo dimensional y roles del DW
`docs/validacion/registro_validacion.md`	13 y 14 — evidencia de las 66 verificaciones
`docs/diagramas/modelo_dimensional.svg`	14 — el diagrama del esquema estrella
`scripts/benchmark/04_explain_analyze_benchmarks.sql`	12 — evidencia de optimización

Antes de escribir una sola cifra

Todos los datos son **sintéticos** (Faker) y el costo por unidad valorativa es un **supuesto paramétrico** de USD 25, declarado así en la columna `fuente` de `academico_oltp.costos_uv`. Declárlo la primera vez que aparezca una cifra en cada capítulo. Lo que el proyecto demuestra es el *método* de cálculo, no el monto.

Capítulo 12 — Diseño e implementación de la base de datos

Extensión orientativa: cuatro a cinco páginas. Es el capítulo más descriptivo de los tres.

Estructura recomendada

Sección	Qué va ahí
Criterios de diseño	Normalización, integridad referencial, reglas en el motor
Esquema operacional	Las 13 tablas por grupos funcionales, con el diagrama entidad-relación
Reglas de negocio en la base	Los dos disparadores, las restricciones CHECK y UNIQUE
Modelo de seguridad	RBAC resuelto en el motor; es el apartado más defendible
Trazabilidad	La bitácora de auditoría y por qué es inescribible para el auditado
Catálogo de costos	La tabla `costos_uv` y su justificación
Optimización	Los `EXPLAIN ANALYZE` de la Fase 1.4

El argumento central del capítulo

Hay un criterio que recorre todo el diseño y conviene enunciarlo temprano, porque unifica decisiones que de otro modo parecen inconexas: **si una regla debe cumplirse siempre, se impone en el motor y no en la aplicación**. Una validación en Python se evade conectándose con `psql`; una restricción de PostgreSQL no.

> PÁRRAFO DE MUESTRA — CRITERIOS DE DISEÑO
>
> El esquema operacional se diseñó bajo un criterio explícito: toda regla que deba cumplirse sin excepción se implementa en el motor de base de datos y no en la capa de aplicación. La diferencia no es estilística. Una validación codificada en la aplicación protege únicamente las operaciones que pasan por ella, de modo que una conexión directa al motor —legítima o no— puede dejar el dato en un estado que el sistema considera imposible. Una restricción declarada en el esquema, en cambio, gobierna cualquier vía de acceso. Bajo ese criterio se implementaron las restricciones de dominio sobre las notas, la unicidad de la inscripción por estudiante y sección, la validación de la ponderación de evaluaciones mediante disparador, y el control de acceso basado en roles.

El modelo de seguridad, que es lo más fuerte que tienes

Tres piezas que hay que explicar en orden: los roles son del motor y no pueden iniciar sesión; la aplicación se conecta como `rol_app`, que es `NOINHERIT`; y cada transacción ejecuta `SET LOCAL ROLE` según el usuario autenticado.

> PÁRRAFO DE MUESTRA — FALLO CERRADO
>
> La aplicación se conecta invariablemente con un rol de servicio declarado `NOINHERIT`. Esa cláusula determina que el rol no adquiera automáticamente los privilegios de los roles a los que pertenece, sino que deba solicitarlos de forma explícita en cada transacción. La consecuencia es que una omisión del programador no degrada en un acceso excesivo sino en un error: si el cambio de rol no se ejecuta, la consulta es rechazada por el motor. El sistema falla cerrado por construcción, y no por disciplina de quien lo programa.

Dos detalles técnicos que el jurado puede repreguntar y conviene que estén en el texto, no solo en tu cabeza:

- **Por qué `SET LOCAL ROLE` y no `SET ROLE`.** `LOCAL` acota el cambio a la transacción; sin él, el rol quedaría asociado a la conexión física y, al reutilizarse desde el *pool*, la siguiente petición heredaría el rol del usuario anterior.
- **Por qué hay una lista blanca de roles en el código.** `SET ROLE` no admite parámetros vinculados, de modo que el nombre del rol debe interpolarse en el texto de la sentencia. La lista blanca impide que ese punto se convierta en un vector de inyección SQL.

La bitácora de auditoría

Vale la pena un apartado propio, porque ilustra un principio de diseño y porque el proyecto resolvió ahí un problema real.

> PÁRRAFO DE MUESTRA — AUDITORÍA
>
> Todo cambio en una calificación queda registrado por un disparador en un esquema de auditoría independiente. El requisito de diseño es que el registro sea inescribible para los roles auditados: una bitácora que el auditado puede modificar no constituye evidencia. Ese requisito entra en conflicto aparente con el funcionamiento del disparador, que por omisión se ejecuta con los privilegios de quien provoca el cambio y, por tanto, carece de permiso para escribir. La solución adoptada fue declarar la función del disparador como `SECURITY DEFINER`, de modo que se ejecute con los privilegios de su propietario. El rastro se registra siempre y ningún rol de la aplicación puede insertarlo, alterarlo ni suprimirlo.

Si quieres un detalle que demuestra dominio: dentro de una función `SECURITY DEFINER`, `CURRENT_USER` pasa a ser el propietario, de modo que el rol efectivo debe registrarse de forma explícita; en caso contrario la bitácora atribuiría todos los cambios al mismo usuario y perdería su utilidad.

Capítulo 13 — Procesos ETL e integración de datos

Extensión orientativa: cinco a seis páginas. Es el capítulo con más sustancia técnica propia.

La arquitectura, en una figura

```
  extracción OLTP  ─┐
                    ├─> transformación ─> verificación de ─> staging ─> carga al
  métricas Neo4j   ─┘                      anonimización               modelo dimensional
      (rol_etl, solo lectura)                                        (rol_dw_carga, escritura)
```

Módulo	Responsabilidad
`conexiones.py`	Punto único de acceso a PostgreSQL y Neo4j
`extraccion.py`	Extrae del OLTP; agrega con `GROUP BY` en el motor
`grafo.py`	Métricas de dependencia curricular desde Neo4j
`transformacion.py`	Limpieza, clasificación y enriquecimiento
`anonimizacion.py`	Verificación en tres barreras antes de que algo salga
`pipeline.py`	Orquesta extracción, transformación y verificación
`carga.py`	Escribe el modelo dimensional en una sola transacción
`validar_dw.py`	66 verificaciones contra fuentes independientes

Anonimización: lista blanca, no lista negra

Este es el apartado que mejor distingue el trabajo, y tiene un argumento que se defiende solo.

> PÁRRAFO DE MUESTRA — ANONIMIZACIÓN
>
> El carné del estudiante llega al proceso ya transformado mediante SHA-256 desde la generación de datos, de manera que el ETL no anonimiza sino que *verifica* que la anonimización se cumple, mediante tres barreras sucesivas. La primera es estructural: una lista blanca enumera las columnas que pueden salir hacia el almacén, y cualquier columna no enumerada detiene el proceso. La elección de una lista blanca en lugar de una lista negra es deliberada; una lista negra solo intercepta los campos que alguien previó, mientras que una lista blanca obliga a modificar explícitamente el control cada vez que se incorpora un atributo nuevo, y es esa modificación la que queda sujeta a revisión. La segunda barrera valida el contenido: el identificador debe presentar la forma de un resumen SHA-256 y ser único. La tercera verifica el aislamiento de las credenciales de aplicación respecto del conjunto analítico.

Limpieza: clasificar, nunca imputar

El otro principio fuerte del pipeline. Ante un dato que no permite calcular un resultado, el ETL deja el resultado nulo y *clasifica el caso*, en lugar de inventar un cero. Un cero imputado a un retiro contaminaría todas las medias.

Resultado de esa clasificación sobre las 36.704 inscripciones:

Clasificación	Inscripciones	Qué significa
APROBADO	24.946	Cerrada con nota igual o superior a 6,00
REPROBADO	8.020	Cerrada por debajo del mínimo
RETIRADO	1.842	No computa como reprobación
SIN_CIERRE	1.896	Aún en curso a la fecha de corte

Esa tabla es el mejor argumento de que el proceso no imputa: las 1.842 retiradas y las 1.896 sin cierre quedan fuera del denominador de la tasa de reprobación, que se calcula sobre las 32.966 inscripciones efectivamente cerradas.

Control de calidad y trazabilidad de la carga

Cada ejecución deja una fila en `dw_academico.carga_control` con sus métricas en JSON. Es material de capítulo: demuestra que el proceso es observable, no una caja negra.

Métrica registrada	Última corrida (10 oct)
Duración de la carga	3,35 segundos
Filas de hechos	36.704
Inscripciones sin costo asignado	0
Inscripciones cerradas con nota incompleta	0
Materias sin nodo en el grafo	ninguna
Nodos del grafo sin materia en el OLTP	ninguno
Materias identificadas como cuello de botella	PRG115, PRG215
Inscripciones no retiradas sin registro de asistencia	22.862

Esa última fila hay que explicarla, no esconderla

22.862 inscripciones no retiradas carecen de registro de asistencia porque el generador de datos sintéticos solo produjo asistencia para los tres primeros de los ocho períodos. Es una limitación del conjunto de datos, no un defecto del proceso: el ETL la detecta, la cuantifica y la deja registrada en lugar de rellenarla. Menciónalo en el capítulo y retómalo en las limitaciones del estudio. Que el proceso *sepa* lo que le falta es un punto a favor.

Separación de privilegios entre lectura y escritura

> PÁRRAFO DE MUESTRA — ROLES DEL PROCESO
>
> El proceso opera con dos roles distintos y mutuamente excluyentes. La extracción se realiza con un rol que posee únicamente privilegio de lectura sobre el esquema operacional, mientras que la carga se ejecuta con un rol que puede escribir en el almacén pero carece de todo acceso al esquema operacional. Ninguno de los dos puede realizar la operación del otro, y ambas restricciones fueron verificadas mediante intentos deliberados de violación que el motor rechazó. La propiedad resultante —que el proceso analítico no pueda modificar el registro académico— no depende de la corrección del código del proceso, sino de los privilegios declarados en el motor.

Capítulo 14 — Modelo dimensional y data warehouse

Extensión orientativa: cuatro a cinco páginas. Abre con el diagrama.

El grano, que es la primera decisión

Antes de hablar de tablas hay que declarar el grano, porque de él depende todo lo demás: **una fila del hecho es una inscripción**, es decir, un estudiante cursando una sección en un período. Son 36.704 filas.

La estructura

Tabla	Filas	Papel
`fact_inscripcion`	36.704	Hecho; 24 columnas entre claves, medidas y banderas
`dim_estudiante`	1.500	Anonimizada: solo el resumen del carné
`dim_seccion`	194	Turno, cupo y número de sección
`dim_materia`	16	Enriquecida con las métricas del grafo
`dim_docente`	20	Escalafón y departamento, sin datos personales
`dim_periodo`	8	Incluye el costo por unidad valorativa vigente
`dim_carrera`	1	Con su departamento, que habilita el data mart
`carga_control`	—	Metadatos y calidad de cada ejecución

Dos decisiones de modelado que conviene justificar porque no son obvias:

- **El costo por unidad valorativa vive en `dim_periodo`**, no en una dimensión propia. Es un atributo del período —cambia con el presupuesto anual— y crear una dimensión de un solo atributo añadiría una unión sin aportar capacidad de análisis.
- **La materia incorpora las métricas del grafo** (`dependientes_totales`, `indice_bloqueo`, `es_cuello_botella`) como atributos dimensionales. Ahí se materializa la integración de los dos motores: una propiedad calculada en Neo4j queda disponible para segmentar hechos relacionales.

Las medidas

Medida	Definición
`costo_inscripcion`	Unidades valorativas × costo por UV del período
`costo_reprobacion`	Lo anterior cuando la inscripción resulta reprobada
`costo_repeticion`	Lo anterior cuando el intento es el segundo o posterior
`nota_final`	Promedio ponderado por el porcentaje de cada evaluación
`porcentaje_asistencia`	Sesiones presente sobre sesiones registradas

Totales de la última corrida, sobre datos sintéticos y con el supuesto de USD 25 por unidad valorativa: costo de matrícula USD 3.670.400 sobre las 36.704 inscripciones, y costo de reprobación USD

802.000 sobre 8.020 inscripciones reprobadas, de las cuales 5.511 corresponden a repeticiones.

Estrategia de carga

> PÁRRAFO DE MUESTRA — CARGA
>
> La carga se implementó como recarga completa dentro de una única transacción: se vacían el hecho y las dimensiones y se repueblan a partir del conjunto transformado. La propiedad que esta estrategia garantiza es la atomicidad frente al consumidor: si la operación falla a mitad de camino, la reversión restituye el almacén al estado anterior, y mientras la carga transcurre el tablero continúa observando la versión íntegra precedente, nunca una combinación de ambas. Con el volumen actual —36.704 hechos cargados en 3,35 segundos— el costo de recargar la totalidad es despreciable frente a la complejidad que introduciría una carga incremental, que exigiría detectar cambios en el origen y conciliar registros preexistentes. La decisión debería revisarse si el volumen creciera en uno o dos órdenes de magnitud.

Ese párrafo hace algo que el jurado valora: justifica la decisión y declara bajo qué condición dejaría de ser válida. Es la diferencia entre defender una elección y defenderla con criterio.

Estrella frente a copo de nieve y Data Vault

Enfoque	Qué aporta	Por qué no se eligió
Estrella	Dimensiones desnormalizadas, menos uniones, consultas simples	**Elegido**
Copo de nieve	Dimensiones normalizadas, menos redundancia	Añade uniones para ahorrar un espacio irrelevante a esta escala
Data Vault	Trazabilidad histórica extensa y múltiples fuentes heterogéneas	Dos fuentes controladas y sin requisito de historiar cambios: la complejidad no se paga

El docente mencionó Data Vault como el enfoque más reciente. No hace falta usarlo, pero sí demostrar que se conoce y que la elección fue razonada, no por desconocimiento.

Validación del almacén

Cierra el capítulo con la evidencia, que es inusualmente sólida: **66 verificaciones, todas correctas**, contrastadas contra fuentes independientes y no contra el propio proceso.

Grupo	Contra qué se contrasta
Completitud	Conteos del esquema operacional
Costo	SQL independiente escrito en el validador, que no reutiliza el ETL
Sistema transaccional	Las mismas funciones de los reportes del coordinador
Correlaciones	`corr()` de PostgreSQL contra pandas
Particiones	Cada segmentación debe repartir exactamente el total
Seguridad y anonimización	Privilegios reales del motor y catálogo del almacén

Que el costo se recalcule con SQL propio y no llamando al ETL es el detalle que convierte la validación en evidencia: si ambos caminos compartieran código, coincidirían aunque los dos estuvieran mal.

Lista de verificación antes de enviar

1. El diagrama del modelo dimensional está incluido y referenciado en el texto.
2. El grano del hecho se declara explícitamente antes de describir las tablas.
3. Se justifica el esquema estrella frente a copo de nieve y Data Vault.
4. Se explica la separación de roles entre lectura del OLTP y escritura del almacén.
5. Se documentan las tres medidas de costo y su cálculo.
6. Se menciona el data mart por departamento.
7. Se declara que los datos son sintéticos y que el costo por UV es un supuesto.
8. La limitación de asistencia en cinco de los ocho períodos está explicada, no omitida.
9. Se cita el registro de validación como evidencia, con su fecha.
10. Formato APA 7 y referencias para el modelado dimensional.

Lo que todavía falta en el repositorio

Pendiente	Impacto en esta entrega
El DDL del almacén está en `database/oltp/` y no en `database/dw/`, que sigue vacío pese a estar declarado en el README	Inconsistencia visible; corregir el README o mover los archivos antes de entregar
Diagrama entidad-relación del modelo operacional	Lo pide el capítulo 11, de la entrega anterior, pero suele revisarse junto con el 12
Visualización de la red curricular (SCRUM-46)	No afecta a esta entrega; sí al capítulo 15

Cifras verificadas en la corrida del 10 de octubre de 2026 sobre el commit `b05136c`, registradas en `docs/validacion/registro_validacion.md`. Diagrama en `docs/diagramas/modelo_dimensional.svg`. Reproducible con `python -m etl.carga` y `python -m etl.validar_dw`.